

### Unit 3

#### Bayes' Theorem and Concept Learning

##### 1. Bayes' Theorem: Definition

Bayes' theorem provides a probabilistic approach to learning by updating beliefs based on observed data. It is defined as:

$$P(h|D) = \frac{P(D|h)P(h)}{P(D)}$$

Where:

- $P(h | D) \rightarrow$  **Posterior Probability**: Probability of hypothesis  $h$  given data  $D$ .
- $P(D | h) \rightarrow$  **Likelihood**: Probability of observing data  $D$  if hypothesis  $h$  is true.
- $P(h) \rightarrow$  **Prior Probability**: Initial belief about  $h$  before seeing data.
- $P(D) \rightarrow$  **Evidence**: Overall probability of data  $D$  across all hypotheses.

##### 2. Bayes' Theorem in Concept Learning

In concept learning, the goal is to determine the most probable hypothesis  $h$  given training data  $D$ .

Using Bayes' theorem, we compute:

$$P(h|D) = \frac{P(D|h)P(h)}{P(D)}$$

- The learner **updates prior knowledge** ( $P(h)$ ) using observed data ( $D$ ) to compute the probability of each hypothesis.
- The **most probable hypothesis** is the one with the highest posterior probability.

##### 3. Maximum A Posteriori (MAP) and Maximum Likelihood Estimation (MLE)

- **MAP Hypothesis:**

$$h_{MAP} = \arg \max_{h \in H} P(D|h)P(h)$$

- Uses both likelihood and prior knowledge.

- **MLE Hypothesis:**

$$h_{MLE} = \arg \max_{h \in H} P(D|h)$$

- Assumes all hypotheses are equally likely (**ignores prior**).

##### 4. Example of Concept Learning with Bayes' Theorem

Consider a hypothesis space  $H$  with three possible hypotheses:

$$H = \{h_1, h_2, h_3\}$$

Given prior probabilities  $P(h_1) = 0.5, P(h_2) = 0.3, P(h_3) = 0.2$ , and observed data  $D$ , we compute:

$$P(h_1|D) = \frac{P(D|h_1)P(h_1)}{P(D)}$$

Whichever hypothesis has the **highest posterior probability** is chosen as the best explanation of the data.

## 5. Importance of Bayes' Theorem in Concept Learning

- Provides a **probabilistic approach** to learning.
- Helps in **handling uncertainty** in training data.
- Enables **incremental learning**, updating beliefs as new data arrives.
- Used in **Bayesian classifiers**, **Naïve Bayes**, and **Bayesian networks**.

### Maximum Likelihood and Least Squares Error Hypotheses

#### 1. Maximum Likelihood Hypothesis (MLE)

The **Maximum Likelihood Estimation (MLE)** principle selects the hypothesis  $h$  that **maximizes the likelihood** of observing the given training data  $D$ .

**Formula:**

$$h_{MLE} = \arg \max_{h \in H} P(D|h)$$

- **Assumption:** All hypotheses are equally likely **before** seeing the data (i.e., uniform prior  $P(h)$ ).
- **Goal:** Choose the hypothesis that makes the observed data most probable.

**Example:**

Suppose we are given coin flip data and want to estimate the probability  $\theta$  of heads. If we observe **8 heads out of 10 flips**, the likelihood is:

$$P(D|\theta) = \theta^8 (1 - \theta)^2$$

Maximizing this likelihood function gives:

$$\theta_{MLE} = \frac{\text{Number of heads}}{\text{Total flips}} = \frac{8}{10} = 0.8$$

Thus, MLE estimates the probability of heads as **0.8**, assuming a **binomial distribution**.

#### 2. Least Squares Error Hypothesis (LSE)

The **Least Squares Error (LSE)** method minimizes the sum of squared differences between predicted values and actual values. It is commonly used in **linear regression**.

**Formula:**

For a hypothesis function  $h(x)$ , LSE minimizes:

$$E = \sum_{i=1}^n (y_i - h(x_i))^2$$

where:

- $y_i$  is the actual target value.
- $h(x_i)$  is the predicted value from the hypothesis.
- $E$  is the **sum of squared errors (SSE)**.

### Example:

For a simple linear model  $h(x) = w_0 + w_1x$ , the best-fit line is obtained by minimizing:

$$E = \sum_{i=1}^n (y_i - (w_0 + w_1x_i))^2$$

This leads to a **closed-form solution** using derivatives or an **iterative gradient descent** approach.

### Relationship Between MLE and LSE:

- If errors follow a **Gaussian (normal) distribution**, MLE and LSE give the same result.
- **MLE is a probabilistic approach**, while **LSE is an optimization approach**.
- LSE assumes errors are **independent and normally distributed** with equal variance.

### Minimum Description Length (MDL) Principle

#### 1. Definition:

The **Minimum Description Length (MDL) principle** is a formalization of **Occam's Razor**, which states that the simplest hypothesis that fits the data should be preferred.

#### MDL Principle:

*"The best hypothesis for a given dataset is the one that minimizes the total length of encoding the hypothesis and the data given the hypothesis."*

Mathematically, MDL selects the hypothesis  $h$  that minimizes:

$$L(h) + L(D | h)$$

Where:

- $L(h)$  = Length of encoding the hypothesis (**model complexity**).
- $L(D | h)$  = Length of encoding the data given the hypothesis (**data fit**).

#### 2. Explanation:

- **A complex hypothesis (large  $L(h)$ )** overfits the data, leading to poor generalization.
- **A too-simple hypothesis (small  $L(h)$ )** may not capture patterns in the data.
- **MDL balances simplicity and accuracy** by penalizing both overly complex and overly simple models.

#### 3. Example:

Consider two models for classifying emails as spam or not spam:

##### 1. **Model A (Complex Rule-Based Model)**

- Requires **100 rules** to classify emails.
- High  $L(h)$  (complex hypothesis).
- Fits training data well but may **overfit**.

## 2. Model B (Simple Naïve Bayes Model)

- Uses **only word frequencies**.
- Lower  $L(h)$  (simpler hypothesis).
- Might not fit training data perfectly but **generalizes better**.

**MDL would prefer Model B** if it provides a good balance between hypothesis complexity and data fit.

## 4. MDL in Machine Learning:

- Used in **decision tree pruning** (e.g., removing unnecessary branches).
- Applied in **Bayesian inference** and **regularization techniques** (e.g., Lasso regression).
- Helps prevent **overfitting** by penalizing complex models.

### Bayes Optimal Classifier

#### 1. Definition:

The **Bayes Optimal Classifier** is the most **probabilistically accurate** classifier, making predictions that **minimize the probability of misclassification**. It considers all possible hypotheses weighted by their probability given the training data.

#### Prediction Rule:

For an input instance  $x$ , the Bayes optimal classifier selects the class label  $y$  that has the highest posterior probability:

$$y = \arg \max_{v \in V} P(v|x, D)$$

Where:

- $V$  is the set of possible class labels.
- $P(v | x, D)$  is the probability that  $x$  belongs to class  $v$ , given data  $D$ .

Using Bayes' theorem:

$$P(v|x, D) = \sum_{h \in H} P(v|x, h)P(h|D)$$

- $P(v | x, h) \rightarrow$  Probability that  $x$  is in class  $v$ , given hypothesis  $h$ .
- $P(h | D) \rightarrow$  Posterior probability of hypothesis  $h$  given data  $D$ .

#### 2. Why is it Optimal?

- **Minimizes expected classification error** by averaging over all possible hypotheses.
- **Uses full probability distribution**, unlike other classifiers that select a single best hypothesis.

### 3. Example: Spam Classification

Suppose we have two hypotheses for classifying an email as **spam (S)** or **not spam (N)**:

- $h_1$ : Emails with “lottery” are spam.
- $h_2$ : Emails from known contacts are not spam.

Given training data  $D$ , we compute:

$$P(S|x, D) = P(S|x, h_1)P(h_1|D) + P(S|x, h_2)P(h_2|D)$$

The Bayes optimal classifier assigns **x to the class with the highest probability**.

### 4. Challenges of Bayes Optimal Classifier:

- **Computationally expensive** – requires summing over all hypotheses.
- **Requires perfect knowledge of prior probabilities**, which is often unavailable.
- **Infeasible for large hypothesis spaces** – approximations like **Naïve Bayes** are used instead.

### 5. Alternative: Naïve Bayes Classifier

- **Naïve Bayes** simplifies the Bayes optimal classifier by assuming **conditional independence** between features.
- Much **faster** and widely used in text classification, spam detection, etc.

## Gibbs Algorithm

### 1. Definition:

The **Gibbs Algorithm** is a **probabilistic learning algorithm** that selects a hypothesis  $h$  randomly from a given **posterior distribution** over the hypothesis space  $H$ . Instead of choosing the single best hypothesis, Gibbs sampling **picks hypotheses probabilistically** based on their likelihood given the training data.

### 2. Steps of the Gibbs Algorithm:

1. Compute the **posterior probability** of each hypothesis  $h \in H$  using **Bayes' theorem**:

$$P(h|D) = \frac{P(D|h)P(h)}{P(D)}$$

2. Randomly select a hypothesis  $h$  **according to its posterior probability**  $P(h | D)$ .
3. Use the selected hypothesis  $h$  to classify new instances.

### 3. Why Use Gibbs Algorithm?

- It is an **approximation** of the **Bayes Optimal Classifier**, which is computationally expensive.
- Instead of averaging over all hypotheses, it selects a single hypothesis based on probability.

- If the **most probable hypothesis is much more likely than others**, Gibbs performs **similarly to the Bayes Optimal Classifier**.

#### 4. Example: Binary Classification Task

- Consider a **spam classifier** with three possible hypotheses:
  - $h_1$ : Emails with "lottery" are spam.
  - $h_2$ : Emails with "offer" are spam.
  - $h_3$ : Emails with "win" are spam.

If we calculate the posterior probabilities:

- $P(h_1 | D) = 0.5$
- $P(h_2 | D) = 0.3$
- $P(h_3 | D) = 0.2$

Then, the **Gibbs Algorithm** randomly selects  $h_1, h_2$ , or  $h_3$  based on these probabilities and classifies emails accordingly.

#### 5. Relationship to Bayes Optimal Classifier

- The **Bayes Optimal Classifier** considers all hypotheses and takes a weighted vote.
- **Gibbs Algorithm picks a single hypothesis probabilistically**, making it more efficient.
- If a hypothesis dominates  $P(h | D)$ , Gibbs performs nearly **as well as Bayes Optimal**.

#### 6. Limitations:

- If multiple hypotheses have similar probabilities, selecting one at random may not be optimal.
- Less stable than deterministic classifiers in noisy environments.

### Naïve Bayes Classifier

#### 1. Definition:

The **Naïve Bayes classifier** is a **probabilistic machine learning model** based on **Bayes' theorem** with the **assumption of conditional independence** among features. It is widely used for **classification tasks**, especially in **text classification, spam filtering, and sentiment analysis**.

#### 2. Bayes' Theorem Recap:

$$P(h|D) = \frac{P(D|h)P(h)}{P(D)}$$

Where:

- $P(h | D) \rightarrow$  Posterior probability of hypothesis  $h$  given data  $D$ .
- $P(D | h) \rightarrow$  Likelihood (probability of data  $D$  given hypothesis  $h$ ).
- $P(h) \rightarrow$  Prior probability of hypothesis  $h$ .
- $P(D) \rightarrow$  Evidence (normalization factor).

### 3. Naïve Bayes Assumption:

The **Naïve Bayes classifier** assumes that all features  $X_1, X_2, \dots, X_n$ , are **conditionally independent** given the class  $C$ :

$$P(X_1, X_2, \dots, X_n | C) = P(X_1 | C) P(X_2 | C) \dots P(X_n | C)$$

Thus, the classification rule becomes:

$$P(C | X_1, X_2, \dots, X_n) \propto P(C) \prod_{i=1}^n P(X_i | C)$$

The class with the highest posterior probability is chosen.

### 4. Steps to Use Naïve Bayes:

1. **Calculate Priors  $P(C)$**  – The probability of each class in the dataset.
2. **Calculate Likelihoods  $P(X_i | C)$**  – The probability of each feature given the class.
3. **Apply Bayes' Rule** – Compute  $P(C | X_1, X_2, \dots, X_n)$ .
4. **Select the class with the highest probability.**

### 5. Example: Spam Email Classification

Dataset:

| Email | Contains "Win" | Contains "Offer" | Spam (S) / Not Spam (N) |
|-------|----------------|------------------|-------------------------|
| 1     | Yes            | No               | Spam (S)                |
| 2     | No             | Yes              | Spam (S)                |
| 3     | No             | No               | Not Spam (N)            |

Step 1: Compute Priors

$$P(S) = \frac{2}{3}, \quad P(N) = \frac{1}{3}$$

Step 2: Compute Likelihoods

$$P(\text{Win} | S) = \frac{1}{2}, \quad P(\text{Win} | N) = 0$$

$$P(\text{Offer} | S) = \frac{1}{2}, \quad P(\text{Offer} | N) = \frac{1}{1}$$

Step 3: Classify a New Email ("Win" and "Offer" present)

Using Naïve Bayes,

$$\begin{aligned} P(S | \text{Win}, \text{Offer}) &\propto P(S) P(\text{Win} | S) P(\text{Offer} | S) \\ &= \frac{2}{3} \times \frac{1}{2} \times \frac{1}{2} = \frac{1}{6} \\ &= P(N | \text{Win}, \text{Offer}) \propto P(N) P(\text{Win} | N) P(\text{Offer} | N) \\ &= \frac{1}{3} \times 0 \times 1 = 0 \end{aligned}$$

Since  $P(S | \text{Win}, \text{Offer}) > P(N | \text{Win}, \text{Offer})$ , the email is classified as **Spam**.

## 6. Advantages of Naïve Bayes:

- **Simple and fast** even for large datasets.
- **Performs well on text classification tasks** like spam detection.
- **Handles missing data** well by assuming independence.

## 7. Limitations of Naïve Bayes:

- **Strong independence assumption** – Often unrealistic in real-world data.
- **Zero probability problem** – If  $P(X_i|C) = 0$ , it wipes out the probability (solved using Laplace smoothing).
- **Not good for continuous numerical features** – Needs extensions like **Gaussian Naïve Bayes**.

## 8. Variants of Naïve Bayes:

- **Multinomial Naïve Bayes** – Used for text classification (e.g., spam detection).
- **Gaussian Naïve Bayes** – Handles continuous data using normal distribution.
- **Bernoulli Naïve Bayes** – Used for binary features (e.g., "word present or not").

## Probably Approximately Correct (PAC) Learning

### 1. Definition:

The **Probably Approximately Correct (PAC) Learning** framework, introduced by Leslie Valiant, provides a **formal way to analyze the feasibility of learning algorithms**. It defines how likely a learning algorithm is to find a hypothesis that is **approximately correct** with high probability.

A hypothesis  $h$  is **probably approximately correct** if:

- It **closely approximates** the true concept  $c$  with **low error** (approximate correctness).
- It does so with **high probability** (probably correct).

### 2. PAC Learning Framework:

A learning algorithm is PAC-learnable if, given:

- **A concept class  $C$**
- **A hypothesis space  $H$**
- **A training dataset  $D$**

It finds a hypothesis  $h$  that satisfies:

$$P(\text{error}(h) \leq \epsilon) \geq 1 - \delta$$

Where:

- $\epsilon$  (**Error Bound**)  $\rightarrow$  The maximum acceptable error in classification.
- $1 - \delta$  (**Confidence Level**)  $\rightarrow$  The probability that the error is within  $\epsilon$ .

This means that with probability at least  $1 - \delta$ , the hypothesis  $h$  has an error of at most  $\epsilon$ .



### 3. Example: PAC Learning in Spam Classification

Suppose we train a spam filter and want to ensure that:

- **At most 5%** of emails are misclassified ( $\epsilon = 0.05$ ).
- **We are 95% confident** in this result ( $\delta = 0.05$ ).

A PAC-learning algorithm must find a hypothesis  $h$  such that the probability of misclassification is at most **5% with 95% confidence**.

### 4. Key Properties of PAC Learning:

- **Efficiency:** If a concept is PAC-learnable, there exists an algorithm that finds a good hypothesis in **polynomial time**.
- **Generalization:** PAC learning ensures the hypothesis generalizes well to unseen examples.
- **Sample Complexity:** The number of training examples needed depends on  $\epsilon$  and  $\delta$ .

### 5. PAC Learning Assumptions:

1. **Random Sampling** – Training examples are drawn **independently and identically** from a fixed distribution.
2. **Fixed Target Concept** – The true concept  $c$  does not change over time.
3. **Computational Feasibility** – The learning algorithm should run in **polynomial time**.

### 6. PAC Learnability of Hypothesis Classes:

- A hypothesis class  $H$  is PAC-learnable if:
  - There exists an algorithm that can find an **approximately correct hypothesis** with **high probability**.
  - The number of training examples required is **polynomial in  $1/\epsilon$  and  $1/\delta$** .

### 7. Relationship to Other Learning Models:

- **VC Dimension:** Determines whether a hypothesis class is PAC-learnable based on its capacity to shatter data points.
- **Occam's Razor & MDL:** Prefers simpler hypotheses, aligning with PAC principles.

### 8. Limitations of PAC Learning:

- **Strong Assumptions** – Real-world data may not be **i.i.d. (independent and identically distributed)**.
- **Computationally Hard for Some Problems** – Some hypothesis classes require exponential time to learn.
- **Noise Sensitivity** – PAC learning struggles with noisy data unless modifications (e.g., noise-tolerant algorithms) are used.

## PAC Learnability

### 1. Definition:

A concept class  $C$  is **PAC-learnable** (Probably Approximately Correct learnable) if a learning algorithm can find a **hypothesis  $h$**  that **closely approximates** the true concept ccc with high probability, using a polynomial number of training examples and computational steps.

A hypothesis  $h$  is **PAC-learnable** if:

$$P(\text{error}(h) \leq \epsilon) \geq 1 - \delta$$

where:

- $\epsilon$  (**Error Bound**)  $\rightarrow$  Maximum acceptable classification error.
- $1 - \delta$  (**Confidence Level**)  $\rightarrow$  Probability that  $h$  has an error at most  $\epsilon$ .

This means that with high probability  $(1 - \delta)$ , the hypothesis  $h$  will have an error less than  $\epsilon$ .

### 2. PAC Learning Framework:

A learning algorithm is PAC-learnable if:

- It can find a hypothesis  $h$  with **low error** in **polynomial time**.
- It requires a **reasonable number of training examples** (polynomial in  $1/\epsilon$  and  $1/\delta$ ).

### 3. PAC Learnability Conditions:

A hypothesis class  $H$  is **PAC-learnable** if:

1. **There exists a polynomial-time algorithm** that finds a hypothesis  $h$  with error  $\leq \epsilon$ .
2. **The number of training examples needed** is polynomial in  $1/\epsilon, 1/\delta$ , and the size of the hypothesis representation.

### 4. PAC Learnability Example: Spam Classification

Suppose we train a spam filter and want to ensure that:

- At most **5% of emails** are misclassified ( $\epsilon = 0.05$ ).
- We are **95% confident** in this result ( $\delta = 0.05$ ).

A PAC-learnable algorithm must find a hypothesis  $h$  that misclassifies at most **5% of emails with 95% probability**.

### 5. Sample Complexity in PAC Learning

To achieve PAC learning, the number of training examples  $m$  required is:

$$m \geq \frac{1}{\epsilon} \left( \log |H| + \log \frac{1}{\delta} \right)$$

where:

- $|H|$  is the size of the hypothesis space.

- $\log \frac{1}{\delta}$  ensures confidence  $(1 - \delta)$ .
- More **complex models require more data**.

## 6. Key Properties of PAC Learning:

- **Efficient Learnability:** PAC learning guarantees efficient hypothesis selection.
- **Generalization:** The hypothesis generalizes well beyond training data.
- **Polynomial-Time Learning:** If a concept is PAC-learnable, it can be learned in polynomial time.

## 7. PAC Learning Assumptions:

1. **Random Sampling** – Training data is drawn **independently and identically (i.i.d.)**.
2. **Fixed Target Concept** – The true concept  $c$  is **unchanging**.
3. **Computational Feasibility** – The algorithm runs in **polynomial time**.

## 8. Relationship to VC Dimension:

- The **Vapnik-Chervonenkis (VC) dimension** determines if a hypothesis class is **PAC-learnable**.
- If the VC-dimension of  $H$  is finite, then  $H$  is PAC-learnable.

## 9. Limitations of PAC Learning:

- **Strong Assumptions** – Data is often **not perfectly i.i.d.** in real-world tasks.
- **Computational Complexity** – Some problems require exponential time.
- **Handling Noise** – PAC learning struggles with noisy data unless modified (**agnostic PAC learning**).

## The Mistake Bound Model for Learning

### 1. Definition:

The **Mistake Bound (MB) Model** is a framework for analyzing learning algorithms based on the **number of mistakes** they make before converging to a correct hypothesis. It is particularly useful for **online learning**, where data is presented sequentially.

A learning algorithm has a **mistake bound  $M$**  if, for any sequence of training examples, it makes at most  $M$  mistakes before it always predicts correctly.

### 2. Key Idea:

- Unlike **PAC learning**, which focuses on probability, the **Mistake Bound Model** measures how many errors a learner makes before learning the correct hypothesis.
- The goal is to **minimize the number of mistakes** while updating the hypothesis.

### 3. Formal Definition:

Let:

- $H$  be the **hypothesis space**.
- The learner is presented with a sequence of labeled examples  $(x_1, y_1), (x_2, y_2), \dots$

The learner predicts  $\hat{y}_i$  for each  $x_i$  and **updates the hypothesis if a mistake is made**.

- The total number of mistakes before convergence is called the **mistake bound  $M$** .

A hypothesis class  $H$  is **learnable in the Mistake Bound model** if there exists an algorithm with a finite mistake bound  $M$ .

### 4. Example: Mistake Bound in Concept Learning

Consider learning a Boolean function like **AND** or **OR**:

- Suppose the true concept is  $y = x_1 \wedge x_2$  (logical AND).
- The learner starts with a general hypothesis and refines it after mistakes.
- If the hypothesis space contains  $n$  attributes, the **mistake bound is at most  $n$** .

### 5. Relationship to Other Learning Models:

#### ✓ Mistake Bound vs. PAC Learning:

- **PAC learning** focuses on error probability in random sampling.
- **MB learning** focuses on the number of mistakes before convergence.
- If a hypothesis class is **PAC-learnable**, it often has a **bounded mistake bound**.

#### ✓ Mistake Bound vs. VC Dimension:

- The **VC dimension** determines sample complexity in PAC learning.
- A low **VC dimension** often implies a low **mistake bound**.

### 6. Applications of the Mistake Bound Model:

- **Online learning algorithms** (e.g., Perceptron for classification).
- **Adversarial learning** (where data is presented in a worst-case order).
- **Learning in dynamic environments** (e.g., stock market predictions).

### 7. Limitations:

- Assumes **mistakes are costly**, but not all mistakes are equally bad.
- Works best in **deterministic environments** (struggles with noisy data).